

Prevention:

- Use `.gitignore` file
- Never commit passwords (use environment variables)
- Keep sensitive data separate

If you did commit:

- Change passwords immediately!
- Rewrite Git history (advanced) or just make repo private

Getting Started with R

RStudio, Projects, and Version Control with Git

Zahid Asghar, School of Economics, QAU

Organised by the SDPI D4D Program

2025-11-20

Welcome!

Today's Focus

- Why R and why it matters for research and data analysis
- Setting up a productive workflow with RStudio
- Understanding R Projects
- Version control basics with Git and GitHub

Why R?

R in Data Analysis and Research

R is purpose-built for statistical computing

Strengths:

- Designed by statisticians for statistics
- Massive ecosystem of specialized packages
- Industry-leading data visualization (ggplot2)
- Active development community
- Excellent reproducibility tools
- Free and open source

Perfect for:

- Data analysis and statistics
- Scientific visualization
- Reproducible research
- Reporting and documentation
- Teaching and collaboration

R vs. Other Languages

Different tools for different jobs:

Language	Best For	Speed	Learning Curve
R	Statistics, data science, visualization	Moderate	Gentle
Python	General programming, ML, automation	Fast	Gentle
Julia	High-performance computing	Very Fast	Moderate
Stata	Economics, panel data	Moderate	Gentle
MATLAB	Engineering, simulations	Fast	Moderate

 Pro Tip

Most researchers use multiple tools. R excels at statistical analysis and creating publication-quality figures.

What You Can Create with R

Visualizations

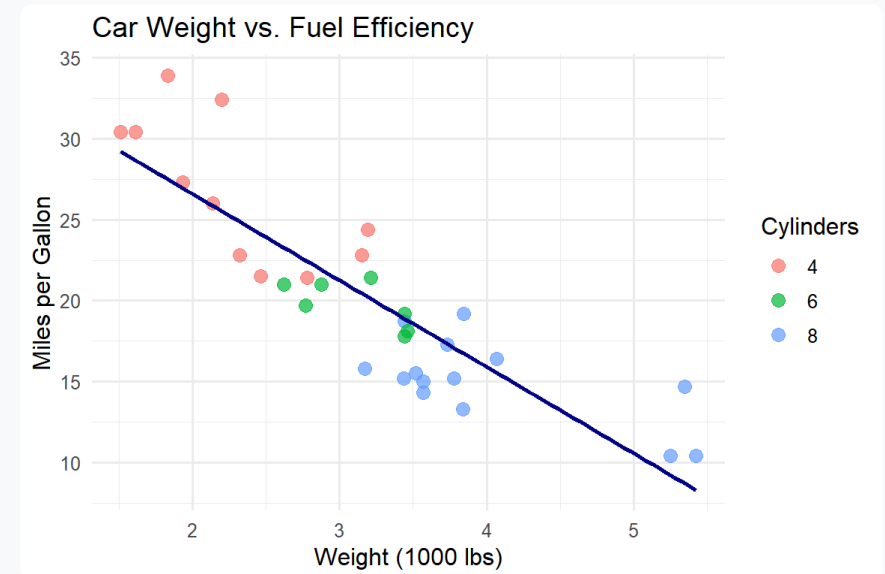
Documents

Analysis

- Publication-quality static graphics (ggplot2)
- Interactive plots (plotly, leaflet)
- Maps and spatial data visualization
- Statistical diagrams and networks

Example: Data Visualization

```
1 library(ggplot2)
2
3 ggplot(mtcars, aes(x = wt, y = mpg)) +
4   geom_point(aes(color = factor(cyl)),
5             size = 3, alpha = 0.7) +
6   geom_smooth(method = "lm",
7             se = FALSE,
8             color = "darkblue") +
9   labs(title = "Car Weight vs. Fuel Efficiency",
10        x = "Weight (1000 lbs)",
11        y = "Miles per Gallon",
12        color = "Cylinders") +
13   theme_minimal(base_size = 12)
```



IDEs for R

What is an IDE?

IDE = Integrated Development Environment

An IDE brings together all the tools you need:

Two Popular IDEs for R

RStudio

The Traditional Choice

- Industry standard since 2011
- Mature and feature-rich
- Excellent for R-specific workflows
- Strong package development tools
- Free and open source
- Great documentation and community

Best for: - Pure R projects - R package development - Teaching R - Traditional workflows

Positron (New!)

The Modern Alternative

- Built by Posit (makers of RStudio)
- Launched in 2024
- Based on VS Code architecture
- Multi-language support (R, Python)
- Modern interface
- Lightning fast
- Still in beta

Best for: - Multi-language projects (R + Python) - Users familiar with VS Code - Modern development workflows - Those wanting cutting-edge features

For This Course

We'll primarily use **RStudio** for demos, but everything works in Positron too! Choose what feels comfortable.

RStudio Interface

Four Main Panes:

1. **Source** (top-left)

- Write and edit scripts
- RMarkdown/Quarto documents

2. **Console** (bottom-left)

- Execute commands
- See output

3. **Environment** (top-right)

- View loaded data/objects
- Command history

4. **Files/Plots/Help** (bottom-right)

- File browser
- Plot viewer
- Package manager
- Help documentation

Note

You can customize pane layout in `Tools > Global Options > Pane Layout`

Getting Positron (Optional)

Want to try the new IDE?

Download:

- Visit github.com/posit-dev/positron
- Click "Releases" on the right
- Download for your OS
- Install like any other app

Key Features:

- Same core functionality as RStudio
- VS Code extensions compatibility
- Better performance
- Modern UI/UX

Interface:

- Similar panes to RStudio
- Command palette (`Ctrl/Cmd + Shift + P`)
- Integrated terminal
- Git support built-in
- Multiple language support

Status:

- Currently in beta
- Actively developed
- Free and open source
- Growing community

Recommendation

Stick with RStudio for learning, explore Positron once comfortable. Both are excellent!

Recommendation

Stick with RStudio for learning, explore Positron once comfortable. Both are excellent!

RStudio vs Positron: Quick Comparison

Feature	RStudio	Positron
Maturity	Stable (10+ years)	Beta (2024+)
Languages	Primarily R	R + Python + more
Learning Curve	Gentle	Gentle (familiar if you know VS Code)
R Package Dev	Excellent tools	Good, improving
Extensions	R-specific	VS Code marketplace
Performance	Good	Faster
Community	Large, established	Growing
Documentation	Extensive	Developing
Best Use Case	Pure R projects	Multi-language projects

Both are made by Posit (formerly RStudio Inc.) and both are free! Your choice depends on your workflow and preferences.

RStudio Tips

Keyboard Shortcuts (save your wrists!)

Action	Windows/Linux	Mac
Run current line	Ctrl + Enter	Cmd + Enter
Assignment operator ←	Alt + -	Option + -
Pipe operator %>%	Ctrl + Shift + M	Cmd + Shift + M
Comment/uncomment	Ctrl + Shift + C	Cmd + Shift + C



Tip

Type Alt + Shift + K (Windows) or Option + Shift + K (Mac) to see all shortcuts

Scripts and Documents

Two Ways to Write Code

R Scripts (.R)

```
1 # Pure code files
2 # Comments with #
3
4 library(tidyverse)
5
6 data ← read_csv("data.csv")
7
8 model ← lm(y ~ x, data = data)
9
10 summary(model)
```

Use for:

Quarto Documents (.qmd)

```
1 # My Analysis
2
3 Here's what I found:
4
5 ```{r}
6 library(tidyverse)
7 data ← read_csv("data.csv")
8 ```
9
10 The results show ...
```

Use for:

- Data processing pipelines
- Function definitions
- Quick analyses

- Reports and papers
- Presentations
- Documented analyses
- Tutorials

Why Quarto/RMarkdown?

Reproducible Research Made Easy

1. **Code + Text in One Place**

- No copy-paste errors
- Analysis and writeup together

2. **Multiple Output Formats**

- PDF (via LaTeX)
- HTML (with interactive elements)
- Word (for collaboration)
- Presentations (like this!)

3. **Truly Reproducible**

- Re-run entire analysis with one click
- Changes automatically propagate
- Perfect for revisions

Quarto Document Structure

```
1 ---
2 title: "My Analysis"
3 author: "Your Name"
4 format: html
5 ---
6
7 ## Introduction
8
9 This study examines ...
10
11 ```{r}
12 #| label: load-data
13 #| echo: false
14
15 library(tidyverse)
16 data <- read_csv("data.csv")
17 ```
18
19 ## Results
20
21 The analysis reveals
```

❗ Important

YAML header (between `---`) controls document metadata and output format

R Projects

The Problem with Working Directories

Don't Do This ❌

```
1 setwd("C:/Users/YourName/Documents/My Research/Project 1/Data")  
2 data ← read.csv("survey.csv")
```

Why this is bad:

- Won't work on anyone else's computer
- Won't work when you move files
- Breaks reproducibility
- Makes collaboration impossible

R Projects to the Rescue

R Projects solve path problems

When you open an `.Rproj` file:

1. RStudio starts fresh R session
2. Working directory automatically set to project folder
3. Use relative paths that work anywhere:

```
1 data ← read_csv("data/survey.csv")  
2 ggsave("figures/plot1.png")
```

4. Project-specific settings saved
5. Easy to zip and share entire project

Creating an R Project

Three common ways:

New Project

Existing Folder

From Git (Best!)

1. `File > New Project > New Directory`
2. Choose project type
3. Select location
4. Click "Create Project"

Organizing Your Project

Recommended structure:

```
my-project/  
├── my-project.Rproj  
├── README.md  
├── data/  
│   ├── raw/  
│   └── processed/  
├── scripts/  
│   ├── 01-clean-data.R  
│   └── 02-analyze.R  
├── documents/  
│   └── paper.qmd  
├── figures/  
└── output/
```

Pro Tips

- Use meaningful names
- Number scripts in order
- Keep raw data separate (never edit!)
- Document everything in README

Version Control with Git

Workshop Schedule Note

Today we're introducing Git/GitHub concepts and basic setup. We'll do a **deep dive into Git workflows** on **Day 2/3** of the workshop with hands-on practice and advanced techniques.

What is Version Control?

Track every change to your project over time

Without version control:

```
thesis_final.docx  
thesis_final_v2.docx  
thesis_final_FINAL.docx  
thesis_final_FINAL_revised.docx  
thesis_final_actual_final.docx
```

- 🤔 Which one is current?
- 🤔 What changed between versions?
- 🤔 Who made what changes?

With version control:

```
thesis.docx
```

- ✅ Complete history saved
- ✅ See all changes
- ✅ Revert to any version
- ✅ Multiple people can work
- ✅ Track who did what

Git Basics

Git = Distributed Version Control System

- Tracks changes to files over time
- Works locally on your computer
- Creates “snapshots” (commits) of your project
- Lets you explore different versions
- Enables collaboration
- Industry standard for code projects

 Note

GitHub vs. Git

Git

- Version control software
- Runs on your computer
- Tracks file changes
- Command-line tool
- Works offline

GitHub

- Online platform
- Built on top of Git
- Cloud storage for repositories
- Web interface
- Collaboration features
- Social coding platform



Tip

Think of Git as your local diary, GitHub as publishing that diary online

Why Use Git for Research?



1. **Never lose work**

- Complete project history
- Restore any previous version
- Experiment safely

2. **Understand what changed**

- See exact changes between versions
- Trace when bugs introduced
- Document decision process

3. **Collaborate effectively**

- Multiple people work simultaneously
- Merge contributions automatically
- Clear attribution of work

4. **Publish and share**

- Make research reproducible

Getting Started with Git

Initial Setup

First time only:

```
1 # Tell Git who you are
2 git config --global user.name "Your Name"
3 git config --global user.email "your.email@example.com"
4
5 # Check settings
6 git config --list
```

Note

Use the same email as your GitHub account!

Create Your First Repository

On GitHub:

1. Go to github.com/new
2. Name your repository (e.g., "test-project")
3. Choose public or private
4. ☒ Check "Add a README file"
5. Click "Create repository"

Important

Initialize with README so repository isn't empty

Clone Repository to Your Computer

In RStudio:

1. `File > New Project > Version Control > Git`
2. Paste repository URL from GitHub
3. Choose where to save
4. Click "Create Project"

You now have:

The Git Workflow

⚠ Today: Overview Only

We're covering Git fundamentals today. **Day 2/3 will include:**

- Detailed workflow demonstrations
- Hands-on practice with your projects
- Troubleshooting common issues
- Advanced collaboration techniques
- Branch management strategies

For now, focus on understanding the concepts!

Four Essential Operations

1. Stage (Add)

Select which changes to save

2. Commit

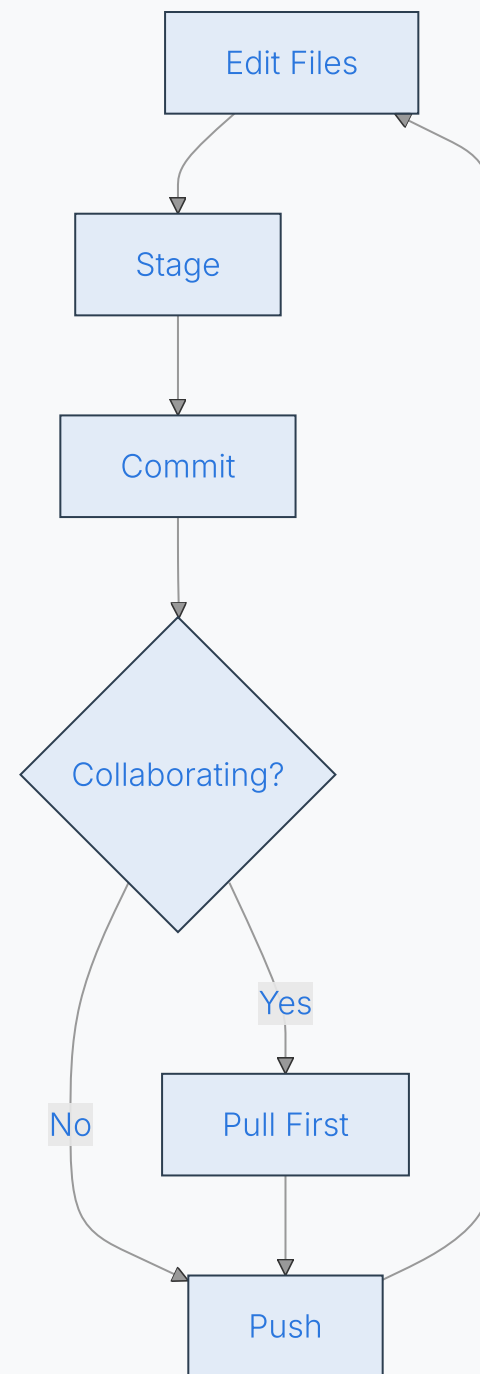
Save staged changes with message

3. Pull

Download changes from GitHub

4. Push

Upload your commits to GitHub



The Commit Cycle

Step 1: Make Changes

Step 2: Stage

Step 3: Commit

Step 4: Push

Edit your files as normal

```
1 # analysis.R
2 library(tidyverse)
3
4 data <- read_csv("data/survey.csv")
5
6 # New analysis
7 model <- lm(outcome ~ treatment, data = data)
```

Writing Good Commit Messages

Bad Messages

```
update  
fixed stuff  
asdf  
final version  
changes  
more changes
```

Problems:

- Not descriptive
- Future you won't understand
- Can't search history

Good Messages

```
Add regression models for main analysis  
  
Fix missing data handling in cleaning script  
  
Update Figure 2 with reviewer comments  
  
Remove deprecated ggplot2 syntax
```

Why better:

- Explains what changed
- Explains why (if needed)
- Searchable
- Professional

RStudio Git Interface

In the Git pane you can:

-  See which files changed (Status column)
-  Stage/unstage files (checkboxes)
-  Commit changes (Commit button)
-  Pull from GitHub (Pull button)
-  Push to GitHub (Push button)
-  Create/switch branches (branch dropdown)
-  View history (History button)
-  Advanced operations (gear menu)

Tip

Keep the Git pane open while working

Important Git Concepts

The Sacred Order

 Always: Stage → Commit → Pull → Push

Never skip the commit before pulling!

1. **Stage** your changes
2. **Commit** with good message
3. **Pull** to get remote changes
4. **Push** your commits

Why this order?

- Pulling after committing preserves your work
- Git can merge changes intelligently
- If conflicts arise, you can resolve them
- Your work is never lost

Merge Conflicts

What causes conflicts?

Two people edit the same lines in the same file

Example scenario:

1. You: Edit line 10 of `analysis.R`, commit
2. Collaborator: Edit line 10 of `analysis.R`, push to GitHub
3. You: Try to push → Conflict!
4. You: Pull → Git says "fix conflicts"
5. You: Open file, see conflict markers:

```
1 <<<<<< HEAD
2 model ← lm(y ~ x1 + x2, data = data) # Your version
3 =====
4 model ← glm(y ~ x1, data = data, family = binomial) # Their version
5 >>>>>> abc123
```

6. You: Choose which to keep (or combine), save, commit, push

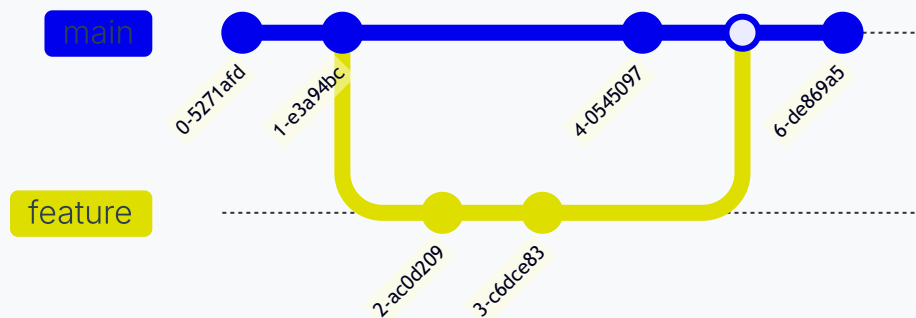
Handling Conflicts

1. Don't Panic
 2. Open the File
 3. Decide What to Keep
 4. Remove Markers
 5. Stage, Commit, Push
-

Conflicts are normal! They're just Git asking you to make a decision.

Branches (Optional but Powerful)

Branches let you work on features independently



- `main` branch = stable version
- Create feature branch for experiments
- Work freely without breaking main
- Merge back when ready (or delete if failed)

Collaboration with GitHub

Inviting Collaborators

For private repositories:

1. Go to repository on GitHub
2. Click "Settings"
3. Click "Collaborators"
4. Click "Add people"
5. Enter their GitHub username
6. They'll receive invitation email

For public repositories:

- Anyone can see and fork
- Control who can directly push

Collaborative Workflow

Good practices:

- ✓ Communicate about who's working on what
- ✓ Pull before starting work
- ✓ Commit small, logical changes
- ✓ Push regularly
- ✓ Write clear commit messages
- ✓ Review each other's code
- ✓ Use branches for big changes

Avoid:

- ✗ Both editing same file simultaneously
- ✗ Committing large files (>100MB)
- ✗ Waiting days between pushes
- ✗ Vague commit messages
- ✗ Pushing broken code to main
- ✗ Force pushing (unless you know why)

Working Solo? Still Use Git!

You're collaborating with future you

- Track your thought process
- Experiment without fear
- Document decisions in commits
- Build portfolio of work
- Practice good habits
- Publish your research code



Tip

Common Git Scenarios

Scenario 1: Made Changes by Mistake

Not Committed Yet

Already Committed, Not Pushed

Already Pushed

Nuclear Option

Easy! Right-click file in Git pane → Revert

Scenario 2: Want to Try Something Risky

Use a branch!

1. Create new branch: `experimental-analysis`
2. Make changes, commit as usual
3. If it works → merge to main
4. If it fails → delete branch

You preserved the working main branch

Scenario 3: Forgot to Pull Before Committing

1. Don't panic
2. Try to pull anyway
3. Git will usually merge automatically
4. If conflicts → resolve them
5. Commit the merge
6. Push

Lesson learned: Pull → Commit → Push

Scenario 4: Accidentally Committed Sensitive Data

Sensitive Data Includes:

- Passwords
- API keys
- Personal information (emails, addresses)
- Confidential data

Advanced Topics (Brief Overview)

.gitignore File

Tell Git to ignore certain files:

```
# R files
.Rproj.user
.Rhistory
.RData
.Ruserdata

# Large data
data/raw/*.csv
data/raw/*.dta

# Outputs (regenerate from code)
figures/*.png
output/*.html

# System files
.DS_Store
Thumbs.db

# Sensitive
config/passwords.txt
.env
```



Tip

Every R project should have a `.gitignore` file

GitHub Features

Issues

- Track bugs
- Plan features
- Discuss ideas
- Assign tasks

Pull Requests

- Propose changes
- Code review
- Discussion
- Quality control

Releases

- Tag versions (v1.0, v2.0)
- Provide downloads
- Document changes

GitHub Pages

- Free website hosting
- Perfect for documentation
- Publish research websites

Git with Large Files

GitHub Limits

- Individual files: 100 MB max
- Repository: 1 GB recommended max

Solutions:

Command Line Git (Optional)

RStudio GUI covers 90% of needs, but CLI is powerful:

```
1 git status          # See what changed
2 git add file.R      # Stage a file
3 git commit -m "message" # Commit
4 git pull            # Pull from remote
5 git push            # Push to remote
6 git log             # View history
7 git diff            # See changes
8 git branch new-feature # Create branch
9 git checkout main    # Switch branch
```

Note

Learn CLI gradually as you need advanced features

Putting It All Together

Your Complete Workflow



1. Setup (once)

- Install R, RStudio, Git
- Create GitHub account
- Configure Git with your info

2. Start Project

- Create repository on GitHub
- Clone to create R Project
- Set up directory structure
- Create .gitignore

3. Daily Work

- Open .Rproj file
- Pull to sync
- Write code, analyze data
- Stage, commit regularly

Best Practices Summary

R Projects

- ✓ Always use R Projects
- ✓ Use relative paths
- ✓ Never use `setwd()`
- ✓ Organize files logically
- ✓ Document in README

Git Commits

- ✓ Commit often
- ✓ Write descriptive messages
- ✓ Commit logical units
- ✓ Pull before pushing
- ✓ Don't commit generated files

Collaboration

- ✓ Communicate clearly
- ✓ Use branches for features
- ✓ Review code together
- ✓ Resolve conflicts promptly
- ✓ Keep main branch stable

Reproducibility

- ✓ Document dependencies
- ✓ Use relative paths
- ✓ Version control everything
- ✓ Make data acquisition reproducible
- ✓ Comment your code

Common Mistakes to Avoid

Resources and Help

R & RStudio

Git & GitHub

Quarto

Getting Help

- RStudio Cheatsheets
- R for Data Science (free online book)
- Posit Community

Practice Exercise (Optional Preview)

 Don't Worry If This Seems Complex!

This is a preview of what we'll practice in detail on Day 2/3. You're welcome to try now, but no pressure!

Complete workflow preview:

1. Create new repository on GitHub called "r-practice"
2. Clone it and create R Project in RStudio
3. Create file `analysis.R` with simple code:

```
1 # My first R project
2 x <- 1:10
3 y <- x^2
4 plot(x, y)
```

4. Stage, commit with message "Add first analysis"
5. Push to GitHub
6. Edit file, add: `print(summary(y))`
7. Commit with message "Add summary statistics"
8. Push again
9. View your commit history on GitHub!



Tip

Alternative: Just focus on creating the GitHub account and installing Git today. We'll practice the workflow together on Day 2/3!

Next Steps

Course Roadmap

What we've covered today:

-  Why R for research
-  RStudio and Positron IDEs
-  R Projects for organization
-  Git/GitHub overview (concepts and setup)

Coming up next:

- **Day 2/3:** Deep dive into Git/GitHub workflows (hands-on practice!)
- Base R fundamentals
- Tidyverse approach
- Data wrangling techniques
- Creating visualizations
- Writing functions
- Reproducible research

Git Learning Path

Today: Understand what Git is and why it matters

Day 2/3: Master Git workflows with guided practice

Beyond: Use Git confidently in your research

Building Good Habits

Start small, be consistent:

- Use R Projects for everything (even small tasks)
- Commit early, commit often
- Write commit messages for future you
- Organize files logically
- Document as you go
- Push regularly (don't lose work!)
- Experiment on branches
- Ask for help when stuck

Your Action Items

! Before Next Session (Day 2/3)

Required:

1.  Install R, RStudio, Git (if not done)
2.  Create GitHub account
3.  Configure Git with your name and email (see slides)

Optional:

4.  Try installing Positron if interested
5.  Complete practice exercise (create repo, clone, commit, push)
6.  Fork and clone course repository
7.  Read Chapter 1-2 of "R for Data Science"

Save for Day 2/3:

- Deep Git/GitHub practice
- Collaborative workflows
- Troubleshooting together

Focus Today

Get your tools installed and accounts created. We'll learn Git by doing on Day 2/3!

Questions?

Let's discuss any questions or concerns

Thank You!

Contact:

-  zahid.quarto.pub
-  zasghar@qau.edu.pk
-  [zahedasghar](https://twitter.com/zahedasghar)
-  github.com/Zahedasghar

Course Materials:

-  github.com/Zahedasghar/d4d

See you next session!